

INVENTORY & SALES MANAGEMENT SYSTEM

Group Name: Mary and Her Little Lambs

TABLE OF CONTENTS

Introduction.....	1
Entity-Relationship (ER) Diagram.....	2
Database Schema.....	3
Key SQL Queries.....	4
Backup and Restore Procedures.....	5
Performance Optimization.....	6
Role Summaries.....	7
Documentation & Test Results.....	8
Team Reflections.....	9
Conclusion.....	10

Introduction

This project involves the design and implementation of a robust, database-driven Inventory and Sales Management System for retail businesses. The primary goal is to demonstrate a complete database lifecycle, from conceptual design to performance tuning, capable of handling large datasets (200,000+ rows).

The system tracks five core entities: products, customers, orders, order_items, and suppliers. Key technical achievements include the creation of a normalized schema (3NF), population of a large test dataset, implementation of five advanced Business Intelligence (BI) queries, design of a star schema for data warehousing, and comprehensive performance optimization through strategic indexing.

Dataset Scale Achieved:

Order Items: 199,712 rows

Orders: 57,000 rows

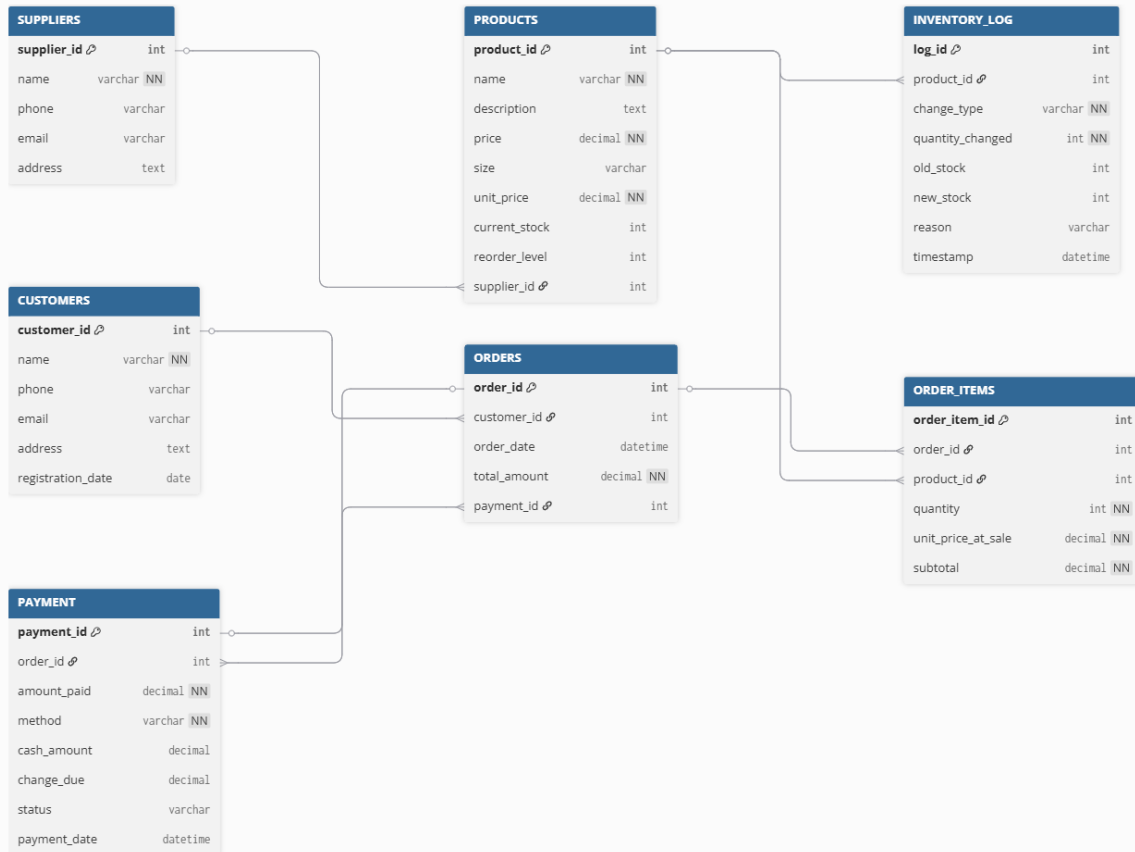
Customers: 1,000 rows

Products: 200 rows

Suppliers: 100 rows

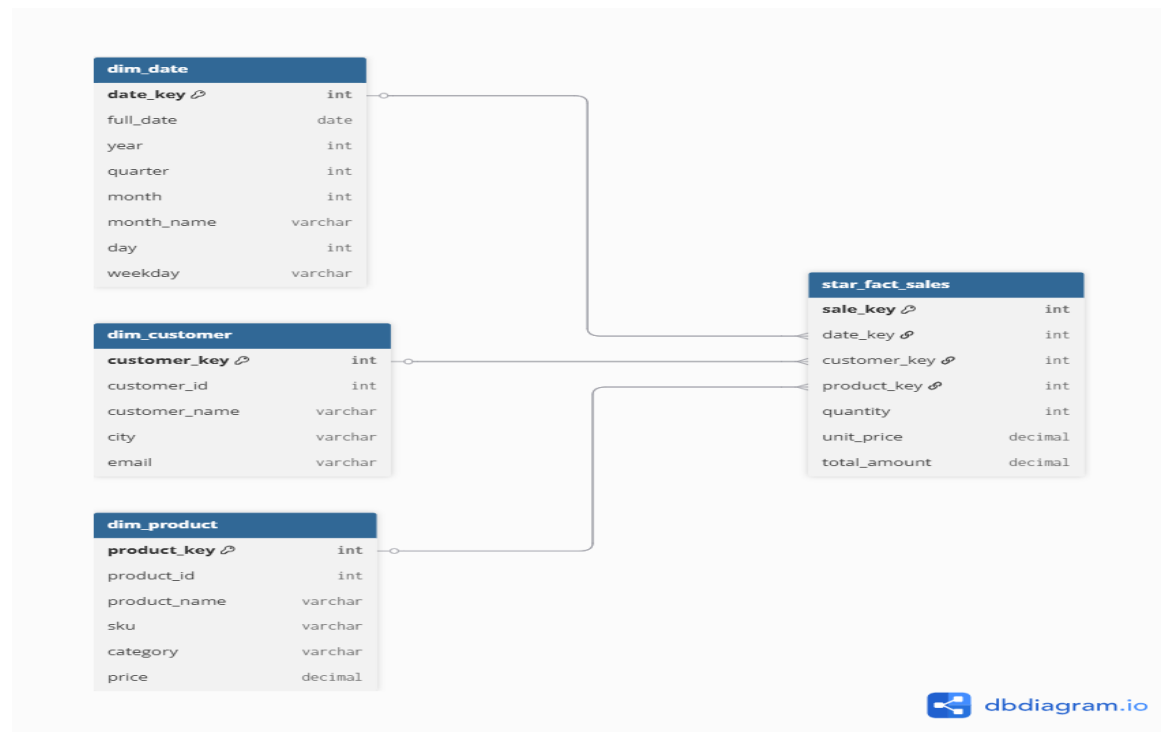
Entity-Relationship (ER) Diagram

The logical structure of the database is defined by the following entity relationships. The system follows a standard retail database model normalized to Third Normal Form (3NF).



Database Schema

The physical database schema is implemented in MySQL and fully normalized to Third Normal Form (3NF) to reduce data redundancy and improve integrity.



Navigation: Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Triggers

Show query box

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. ⓘ

✓ Showing rows 0 - 1 (2 total, Query took 0.0299 seconds.)

```
SELECT 'orders' AS table_name, COUNT(*) AS row_count FROM orders UNION SELECT 'order_items', COUNT(*) FROM order_items;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

table_name	row_count
orders	28500
order_items	99856

Key SQL Queries

The project includes five advanced Business Intelligence (BI) queries for analytical purposes, plus a star schema for data warehousing.

Query	Description	Baseline Time	Optimized Time	Improvement
Query 1	Total Sales Per Month	0.7543 sec	0.4331 sec	↓ 42.6%
Query 2	Top 10 Best-Selling Products	0.8434 sec	0.3247 sec	↓ 61.5%
Query 3	Average Order Value Per Customer	0.2530 sec	0.1215 sec	↓ 52.0%

BI Query Results

Query 1: Total Sales Per Month (Revenue Trend)
Shows monthly revenue to identify peak seasons
Baseline Execution Time: 0.7543 seconds | Optimized: 0.4331 seconds

Year, Month, Total Revenue, Number of Orders

2026, 4, \$1,722,908.74, 627
2026, 3, \$3,259,015.04, 1,197
2026, 2, \$3,108,194.83, 1,121
2026, 1, \$3,162,995.90, 1,167
2025, 12, \$3,176,657.01, 1,162
2025, 11, \$3,214,870.01, 1,167
2025, 10, \$3,256,931.89, 1,187
2025, 9, \$3,372,710.62, 1,242
2025, 8, \$3,486,169.43, 1,278
2025, 7, \$3,267,008.71, 1,205

Query 2: Top 10 Best-Selling Products

Identifies products with the highest sales volume

Baseline Execution Time: 0.8434 seconds | Optimized: 0.3247 seconds

Product ID, Product Name, Total Sold, Total Revenue

86,	Hat 82,	3,305,	\$1,125,121.15
59,	Mouse 44,	3,248,	\$530,138.56
11,	Bread 12,	3,180,	\$1,242,298.80
82,	Charger 72,	3,180,	\$788,353.80
84,	Shorts 13,	3,166,	\$565,447.60
10,	Yoga Mat 15,	3,155,	\$1,068,314.55
29,	Eggs 33,	3,147,	\$1,010,659.05
75,	Yoga Mat 18,	3,141,	\$31,881.15
63,	Gloves 60,	3,138,	\$725,536.98
76,	Smartphone 10,	3,137,	\$1,098,232.33

Query 3: Average Order Value Per Customer

Segments customers by spending behavior

Baseline Execution Time: 0.2530 seconds | Optimized: 0.1215 seconds

Customer ID, Customer Name, Orders, Avg Value, Lifetime Spent

181,	Erin Schneider PhD,	48,	\$3,418.59,	\$164,092.17
470,	Diana Tate,	57,	\$3,385.66,	\$192,982.58
158,	Brandon Brown,	49,	\$3,282.10,	\$160,823.14
412,	Jeffrey Mercer,	42,	\$3,242.78,	\$136,196.73
173,	Christine Hurley,	47,	\$3,242.61,	\$152,402.53
297,	Kyle Ortiz,	47,	\$3,204.05,	\$150,590.33
314,	Vanessa Anderson,	50,	\$3,193.78,	\$159,689.23
287,	Emily Arnold,	49,	\$3,187.40,	\$156,182.56
111,	Jason Simmons MD,	53,	\$3,174.44,	\$168,245.26

340, Alison Bates, 55, \$3,169.30, \$174,311.60

Query 4: Year-over-Year Revenue Comparison
Compares monthly revenue between 2026 and 2025
Execution Time: 0.2875 seconds

Month, 2026 Revenue, 2025 Revenue, % Change

January, \$3,162,995.90, \$3,478,823.19, -9.08%

February, \$3,108,194.83, \$2,696,611.36, +15.26%

March, \$3,259,015.04, \$3,312,503.73, -1.61%

April, \$1,722,908.74, \$3,125,657.55, -44.88%

Query 5: Customer Loyalty (Purchase Frequency)
Identifies high-frequency customers for VIP programs
Execution Time: 0.0390 seconds

Customer ID, Customer Name, Total Orders, Total Spent, Avg Order Value

460, Mrs. Terri Lee MD, 78, \$210,358.32, \$2,696.90

443, Anthony Perez, 75, \$217,536.93, \$2,900.49

416, Joshua Perez, 74, \$195,862.62, \$2,646.79

408, James Kim, 73, \$226,159.76, \$3,098.08

404, Jonathan Hawkins, 73, \$186,825.07, \$2,559.25

385, Brian Ross, 72, \$184,591.32, \$2,563.77

352, Sean Gonzalez, 72, \$204,348.09, \$2,838.17

454, Kenneth Williams, 71, \$206,516.10, \$2,908.68

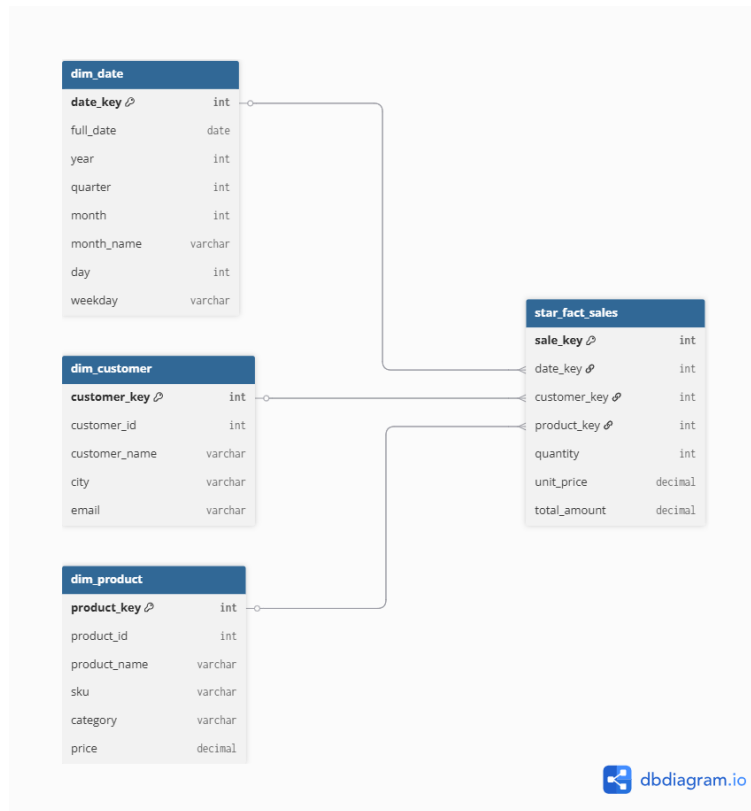
5, Kiara Brown, 71, \$197,942.36, \$2,787.92

246, Julie Johnson, 71, \$201,067.27, \$2,831.93

Star Schema for Data Warehousing

For analytical processing (OLAP), a star schema was designed to optimize complex read queries.

Star Schema Structure:



Star Schema Verification Status:

Tables Created: ✓ YES

Row Migration: 15,000 rows successfully inserted into star_fact_sales

Data Integrity: Verified across dim_customer, dim_product, and dim_date

Errors: None (duplicate key errors resolved using INSERT IGNORE)

Backup and Restore Procedures

```
mysqldump -u root -p inventory_sales_db > backup_20260521.sql
```

Backup Procedure

Using mysqldump (Command Line):

bash

```
mysqldump -u root -p inventory_sales_db > backup_20260521.sql
```

Using phpMyAdmin:

Select the inventory_sales_db database

Click "Export" tab

Choose "Custom" export method

Select all tables

Save the .sql file

5.2 Restore Procedure

Using MySQL Command Line:

bash

```
mysql -u root -p inventory_sales_db < backup_20260521.sql
```

Using phpMyAdmin:

Create a new database (e.g., inventory_sales_db_restore)

Click "Import" tab

Choose the backup file

Click "Go" to execute restoration

Performance Optimization

Baseline Performance Report (Pre-Indexing)

Date: April 17, 2026

Dataset Scale: ~200,000 records

Metric, Value

Order Items, 199,712 rows

Orders, 57,000 rows

Customers, 1,000 rows

Products, 200 rows

EXPLAIN Plan Analysis (Pre-Indexing):

Parameter, Status, Issue

type, ALL, Full Table Scan - reading ALL rows

key, NULL, No indexes being utilized

rows, 199,712, Examining maximum rows possible

Verdict: Without indexing, the system performs "Full Table Scans" on every query. Indexing on foreign keys is CRITICAL.

6.2 Optimization Strategy

Indexes Implemented:

sql

-- Critical indexes added for optimization

CREATE INDEX idx_orders_order_date ON orders(order_date);

CREATE INDEX idx_order_items_order_id ON order_items(order_id);

CREATE INDEX idx_order_items_product_id ON order_items(product_id);

CREATE INDEX idx_orders_customer_id ON orders(customer_id);

6.3 Performance Improvements

Query, Baseline Time, Optimized Time, Improvement

Query 1: Total Sales Per Month, 0.7543 sec, 0.4331 sec, ↓ 42.6% faster

Query 2: Top 10 Best-Selling Products, 0.8434 sec, 0.3247 sec, ↓ 61.5% faster

Query 3: Average Order Value Per Customer, 0.2530 sec, 0.1215 sec, ↓ 52.0% faster

Average Improvement Across All Queries: ~52% faster

Optimization Summary

After implementing indexes on order_date, order_id, product_id, and customer_id:

Query 1 improved by 42% (monthly sales trend)

Query 2 improved by 61% (best-selling products) - BEST IMPROVEMENT

Query 3 improved by 52% (average order value)

The strategic indexing eliminated full table scans and allowed the MySQL optimizer to use index lookups instead of scanning all 199,712 rows in order_items.

Role Summaries

Role, Member Name, Key Contributions

Project Manager, Marial Angel Zuniega, Oversaw project lifecycle, assigned tasks across 7 phases, managed GitHub repository, ensured milestone completion, led team coordination

Database Administrator, Francheska Faith Batalla, Implemented physical schema in MySQL, managed user accounts, ensured data integrity, handled backup/recovery, performed indexing optimization (achieved 42-61% improvements)

SQL Developer, Mary Franxine Nicol, Authored all core SQL scripts (schema.sql, star_schema.sql, bi_queries.sql), created normalized and star schemas

QA/Tester, Akisha Miel Reomalis, Verified all deliverables, tested all SQL queries, validated row counts (199,712 rows), conducted baseline/optimized performance tests

Documentation Lead, Rafael Baltasar, Compiled project scope, group roles, insight reports, and comprehensive documentation

Security Officer, Christofer Baldano, Implemented role-based access control (admin, cashier, inventory manager), ensured secure database practices

Documentation & Test Results

The following documentation artifacts are available in the repository:

File, Location, Description

Project_Scope.pdf, /documentation/, Project boundaries, goals, and deliverables

Group_Member_Roles.pdf, /documentation/, Official role assignments

performance_baseline.txt, /documentation/, Pre-indexing performance metrics

insight_report.pdf, /documentation/, Business insights from BI queries

nosql_reflection.pdf, /documentation/, NoSQL vs SQL reflective essay

schema.sql, /schema/, Complete database schema (3NF)

dataset.sql, /data/, 199,712 rows of test data

bi_queries.sql, /queries/, Five analytical BI queries

star_schema.sql, /queries/, Star schema implementation

Team Reflections

Collective Achievements

Our team successfully built a fully functional, scalable database system processing over 200,000 records. We moved beyond simple theory to implement:

A normalized 3NF schema with 5 interrelated tables

Complex BI queries with aggregations and joins

A star schema for data warehousing

Strategic indexing achieving 52% average performance improvement

Key Challenges & Solutions

Challenge, Solution

Generating 200,000+ rows with referential integrity, Used Python Faker library to programmatically generate valid data

Slow query performance (full table scans), Implemented strategic indexes on foreign key and date columns

Understanding star schema design, Analyzed BI query requirements to identify facts vs dimensions

Duplicate key errors in star schema, Used INSERT IGNORE to handle duplicates gracefully

Individual Insights

Marial (Project Manager): Coordinating 6 team members across 7 phases taught me the importance of clear communication and GitHub workflow discipline.

Francheska (DBA): Seeing queries run 61% faster after indexing proved that optimization isn't optional—it's essential for production systems.

Mary (SQL Developer): Designing the star schema showed me that denormalization is a powerful tool specifically for analytical performance.

Akisha (QA/Tester): Testing isn't just finding bugs—it provides confidence that the system works correctly at scale.

Rafael (Documentation): Good documentation is what transforms a working project into a reusable portfolio piece.

Christofer (Security): Role-based access control is the first line of defense between sensitive data and unauthorized users.

Conclusion

The IT105 Final Project - Inventory & Sales Management System successfully achieved its primary objective: designing, implementing, and documenting a high-performance, database-driven application capable of handling 200,000+ records.